

VPS Deployment Playbook

CONTENTS

[Step 1 — DNS](#)

[Step 2 — Folder structure](#)

[Step 3 — Docker image](#)

[Step 4 — docker-compose.yml](#)

[Step 5 — nginx server block](#)

[Step 6 — SSL certificate](#)

[Webroot \(preferred — no downtime\)](#)

[Standalone \(only if webroot is unavailable\)](#)

[Step 7 — Start everything](#)

[Step 8 — Future deployments](#)

[Cheatsheet](#)

[The checklist for any new site](#)

[Notes on this server](#)

[The renewal cron](#)

Server: `root@209.145.55.76` · **SSH key:** `C:\Users\PCL\.ssh\orion_vps`

The repeatable process for putting any new project on this box. Steps 1-3 are one-time setup per project; step 8 is the only one with a tricky ordering.

Step 1 — DNS

Do this first: it takes 5-30 minutes to propagate, and everything after it depends on the name resolving.

In the domain registrar (or Cloudflare), add an A record:

FIELD	VALUE
Type	A
Name	@ for the root domain, or the subdomain (e.g. hermes)
Value	209.145.55.76
TTL	Auto, or 300

Verify it has propagated **before touching nginx or SSL**:

```
nslookup yourdomain.com
# must return 209.145.55.76
```

Step 2 — Folder structure

```
ssh -i C:\Users\PCL\.ssh\orion_vps root@209.145.55.76

mkdir -p /opt/yourproject
cd /opt/yourproject
```

Every project lives in its own `/opt/projectname/`:

```
/opt/yourproject/
├─ docker-compose.yml
├─ .env                ← secrets, never committed
└─ nginx.conf          ← only if the project has its own nginx
```

Step 3 — Docker image

Option A — build locally, push to Docker Hub (recommended)

```
docker build -t yaseenishfaq/yourproject:latest .
docker push yaseenishfaq/yourproject:latest
```

Then on the VPS: `docker pull yaseenishfaq/yourproject:latest`

Option B — build on the VPS

```
cd /opt/yourproject
docker build -t yourproject:latest .
```

Step 4 — docker-compose.yml

At `/opt/yourproject/docker-compose.yml`:

```
services:
  app:
    image: yaseenishfaq/yourproject:latest # or the local image name
    container_name: yourproject-app
    restart: unless-stopped
    env_file: .env
    expose:
      - "3000" # internal only, NOT published
    networks:
      - scalaro-net # share Scalaro's nginx network

networks:
  scalaro-net:
    external: true
```

Why `scalaro-net`. Scalaro's nginx container is already on this network. Joining it makes the new app reachable from that nginx **by container name**, with no port published on the host — so nothing can collide with the fifteen other applications already running here, and nothing is exposed to the internet except through nginx.

Check the exact network name first:

```
docker network ls | grep scalaro
```

Step 5 — nginx server block

Edit the shared config at `/opt/scalaro/nginx.conf` and add:

```
# HTTP → HTTPS
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    location /.well-known/acme-challenge/ { root /etc/letsencrypt/www; }
    location / { return 301 https://$server_name$request_uri; }
}

# HTTPS
server {
    listen 443 ssl http2;
    server_name yourdomain.com www.yourdomain.com;

    ssl_certificate      /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/yourdomain.com/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers          HIGH:!aNULL:!MD5;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;

    location / {
        proxy_pass http://yourproject-app:3000; # container name : port
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Test before reloading:

```
cd /opt/scalaro && docker compose exec nginx nginx -t
```

Step 6 — SSL certificate

Get the certificate before the HTTPS block is active — nginx will not start with an `ssl_certificate` path that does not exist yet.

Webroot (preferred — no downtime)

The ACME challenge location is already in the nginx blocks above, so:

```
certbot certonly --webroot -w /etc/letsencrypt/www \
-d yourdomain.com -d www.yourdomain.com \
--email yasinishfaq5@gmail.com --agree-tos --non-interactive
```

Standalone (only if webroot is unavailable)

Binds port 80 itself, so nginx has to stop first — which interrupts **every** site on the box:

```
cd /opt/scalaro && docker compose stop nginx
certbot certonly --standalone -d yourdomain.com --email yasinishfaq5@gmail.com --agree-tos --non-interactive
cd /opt/scalaro && docker compose start nginx
```

The certificate lands in `/etc/letsencrypt/live/yourdomain.com/`.

Verify renewal works:

```
certbot renew --dry-run
```

Step 7 — Start everything

```
# 1. the app
cd /opt/yourproject
docker compose up -d
docker compose ps
docker compose logs -f --tail=50

# 2. pick up the new server block
cd /opt/scalaro
docker compose exec nginx nginx -t && docker compose exec nginx nginx -s reload
```

Test:

```
curl -I https://yourdomain.com    # expect HTTP/2 200
```

Step 8 — Future deployments

```
# local
docker build -t yaseenishfaq/yourproject:latest .
docker push yaseenishfaq/yourproject:latest
```

```
# VPS
ssh -i C:\Users\PCL\.ssh\orion_vps root@209.145.55.76 \
"cd /opt/yourproject && docker compose pull && docker compose up -d"
```

Or a `deploy.sh` locally:

```
#!/bin/bash
set -e
PROJECT=yourproject
IMAGE=yaseenishfaq/$PROJECT:latest

echo "Building..."
docker build -t $IMAGE .

echo "Pushing..."
docker push $IMAGE

echo "Deploying..."
ssh -i ~/.ssh/orion_vps root@209.145.55.76 \
"cd /opt/$PROJECT && docker compose pull && docker compose up -d && echo Done"
```

Cheatsheet

TASK	COMMAND (ON THE VPS)
All running containers	<code>docker ps</code>
Logs for a project	<code>cd /opt/yourproject && docker compose logs -f</code>
Restart a project	<code>cd /opt/yourproject && docker compose restart</code>
Stop a project	<code>cd /opt/yourproject && docker compose down</code>
Reload nginx	<code>cd /opt/scalaro && docker compose exec nginx nginx -s reload</code>
Test nginx config	<code>cd /opt/scalaro && docker compose exec nginx nginx -t</code>
Disk space	<code>df -h</code>
Networks	<code>docker network ls</code>

The checklist for any new site

- ☐ 1. DNS A record → 209.145.55.76
- ☐ 2. nslookup confirms it propagated
- ☐ 3. /opt/yourproject/ created
- ☐ 4. docker-compose.yml written, joining scalaro-net
- ☐ 5. .env created with the secrets
- ☐ 6. image pushed to Docker Hub (or built on the VPS)
- ☐ 7. docker compose up -d
- ☐ 8. SSL certificate obtained with certbot
- ☐ 9. server block added to /opt/scalaro/nginx.conf
- ☐ 10. nginx -t → nginx -s reload
- ☐ 11. curl -I https://yourdomain.com ✓

Notes on this server

nginx runs in a container (`scalaro-nginx-1`) holding ports 80 and 443. The host's own nginx is installed but serves nothing — a file dropped in `/etc/nginx/sites-enabled/` will appear to work and do nothing.

Certificates live at `/etc/letsencrypt` on the host, mounted into the nginx container. There are two webroot paths already in use across the config: `/etc/letsencrypt/www` and `/var/www/certbot`. Prefer `/etc/letsencrypt/www` for anything new — it is the one already mounted into the container.

Ports already taken: 80, 443, 3001, 4000, 5432, 5433, 8010, 8080, 8085, 8090, 8092, 8095, 8096, 8123. Joining `scalaro-net` avoids the question entirely, which is why the playbook does it that way.

The renewal cron needs fixing — see below.

The renewal cron

```
30 2,14 * * * certbot renew --standalone --quiet
```

The certificates on this box were issued with the **webroot** authenticator, and `certbot renew` would normally reuse each certificate's own recorded method. Passing `--standalone` on the command line **overrides** that and forces certbot to bind port 80 — which `scalaro-nginx-1` is holding. The renewal will fail.

Nothing has broken yet only because no certificate has come due: Let's Encrypt renews at 30 days remaining, and the earliest here is 64 days out. The first real attempt is about a month away.

The fix is to drop the flag so each certificate renews the way it was issued:

```
crontab -e  
# change to:  
30 2,14 * * * certbot renew --quiet 2>&1 | logger -t certbot
```

Then confirm:

```
certbot renew --dry-run
```